

C²AFE: Capacity Curve Annotation and Feature Extraction

Cesar Gomes

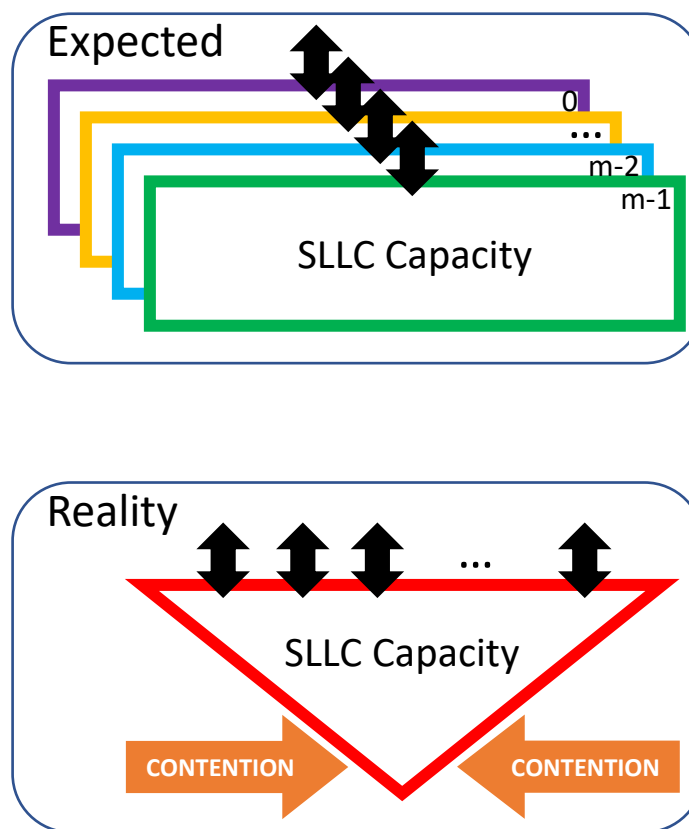
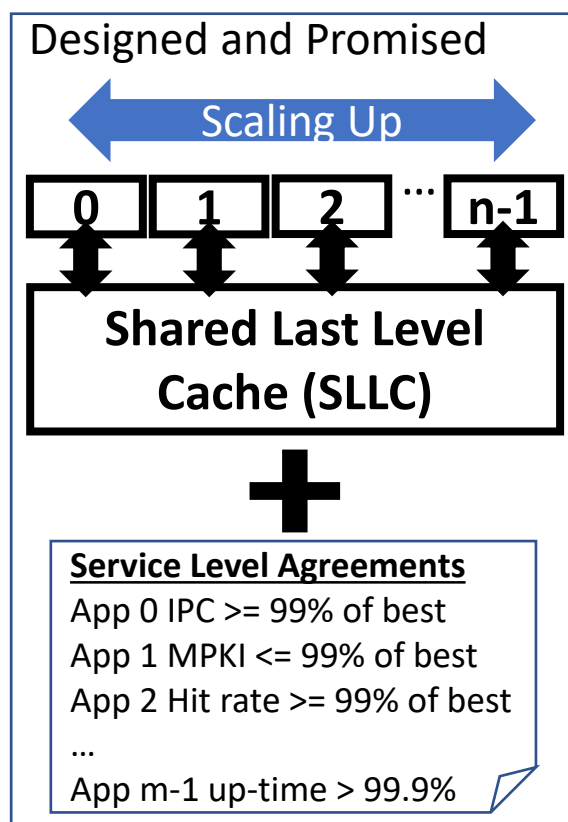
Mark Hempstead

Department of Electrical and Computer Engineering

School of Engineering

Tufts University

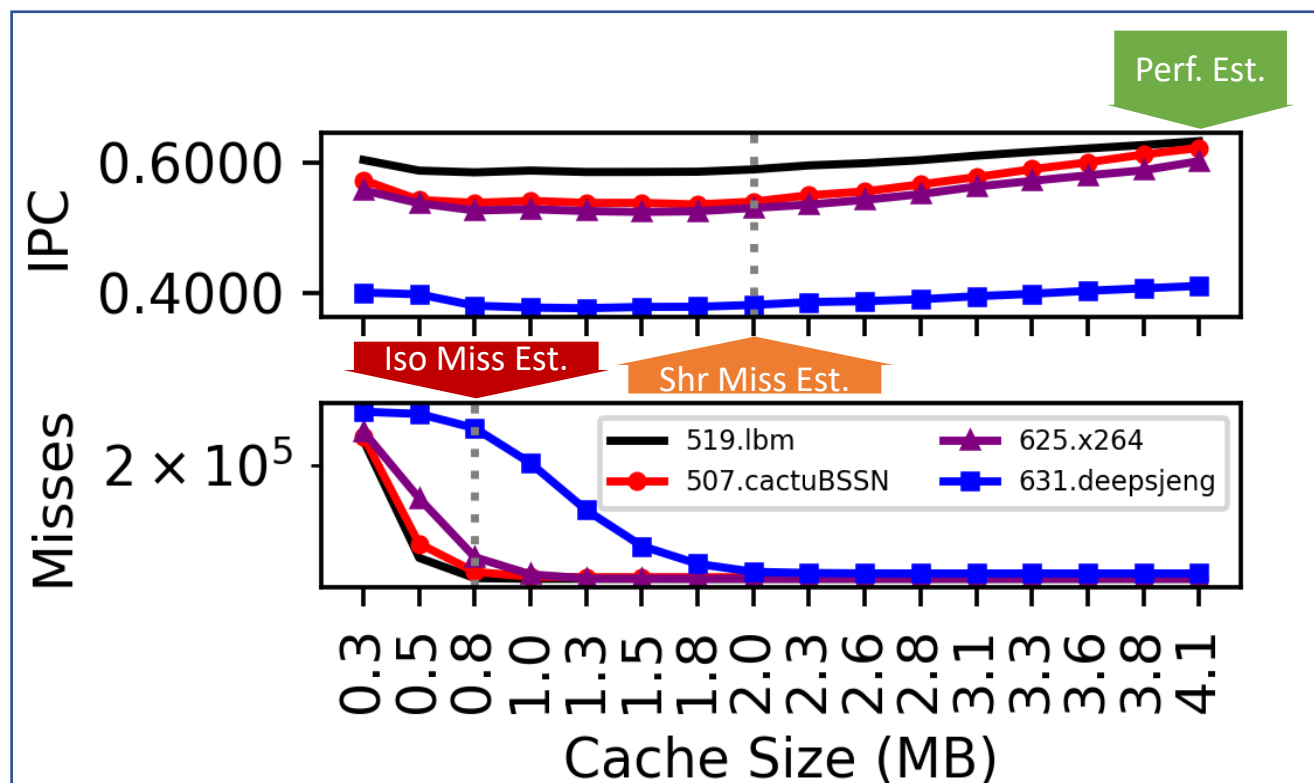
Shared caches see increasing pressure due to contention from Core Scaling and Performance Guarantees



- Scaling cores, highly utilized processors [1], and performance agreements [2] put pressure on an already scarce and highly contended resource: Shared Cache.
- Modern techniques often profile cores/threads/applications **individually** to achieve these goals, with performance agreements imposing the need to preserve behavior **from** contention [3,5,6,9].

Q. We want to preserve behavior, right?

Yes, but profilers estimate isolation behavior with misses

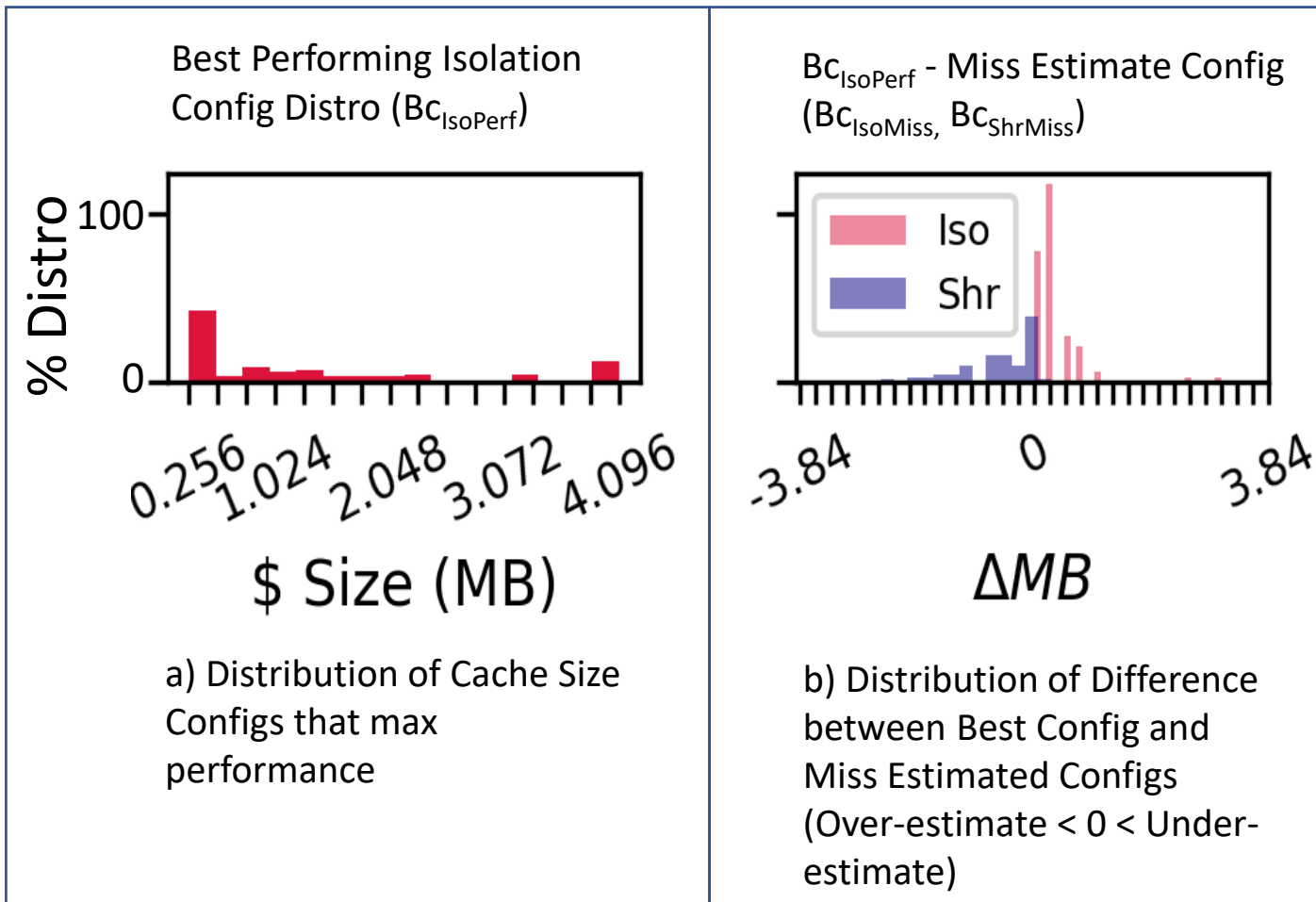


519.lbm Performance and Miss Curves In isolation and Shared Contexts:
Performance varies differently from misses

- Isolation (Iso) misses become stable at 1MB.
- Performance also stabilizes up to 2MB.
- One Shared (Shr) miss curve (631.deepsjeng) becomes stable after 2MB.
- Performance increases after 2MB, but misses remain the same.
- **TAKEAWAY:** The Iso miss curve ignores performance behavior at small configs, and under-estimates cache needed; Shr miss curve offers closer estimation.

Q: How much do isolation misses under-estimate?

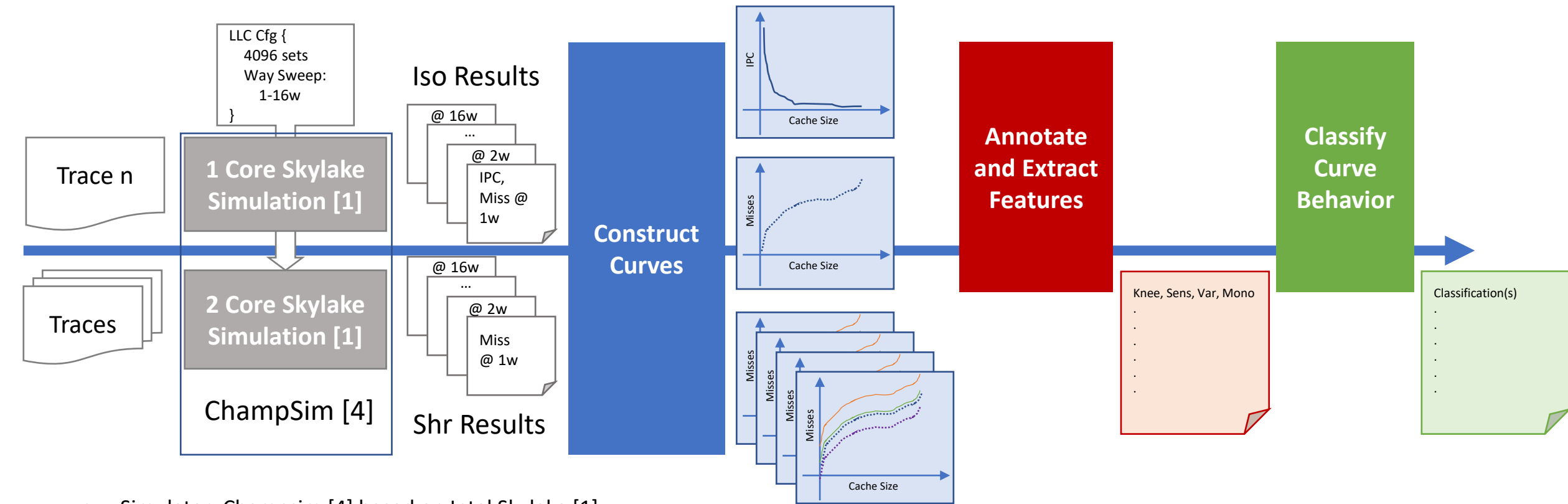
Isolation Misses Underestimate Caching Needs



- Choosing cache size based on maximizing performance yields diverse solutions through cache configs ($Bc_{IsoPerf}$) (figure a).
- Choosing cache size based on minimizing Isolation under-estimates cache needs, while choosing based on minimizing Shared Miss (Shr) over-estimates cache needs (figure b).
- **TAKEAWAY:** Shared Misses have a HIGHER likelihood to estimate cache needs in comparison to Isolation Misses.

Q: How can we investigate this deeper?

Capacity Curve (C^2) Annotation and Feature Extraction - Overview

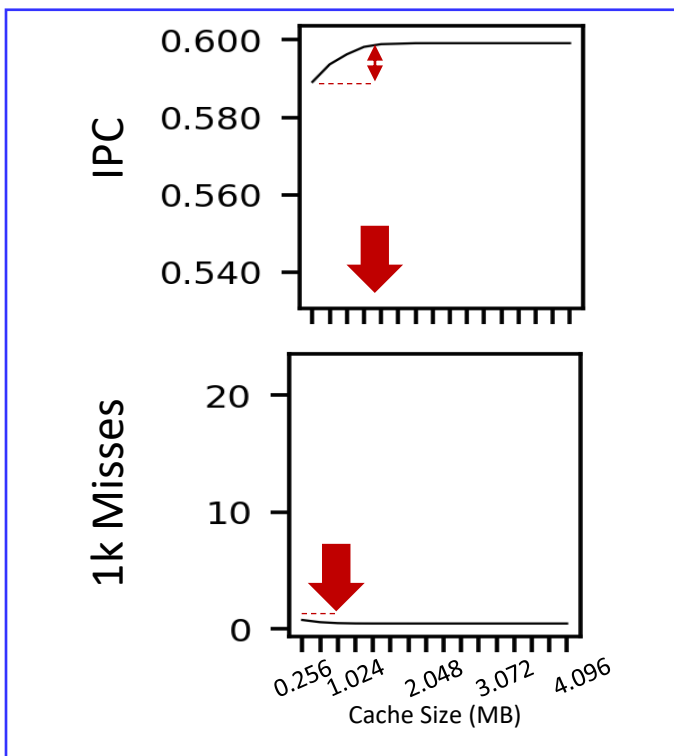


- Simulator: Champsim [4] based on Intel Skylake [1]
- Traces: SPEC 06, SPEC 17 Rate and Speed [7], and Problem Based Benchmark Suite (PBBS) [8]
- Trace Generation Method: Skip 1 Billion Instructions, Trace 6 million
- Simulation Method: All traces run simultaneously with every other trace

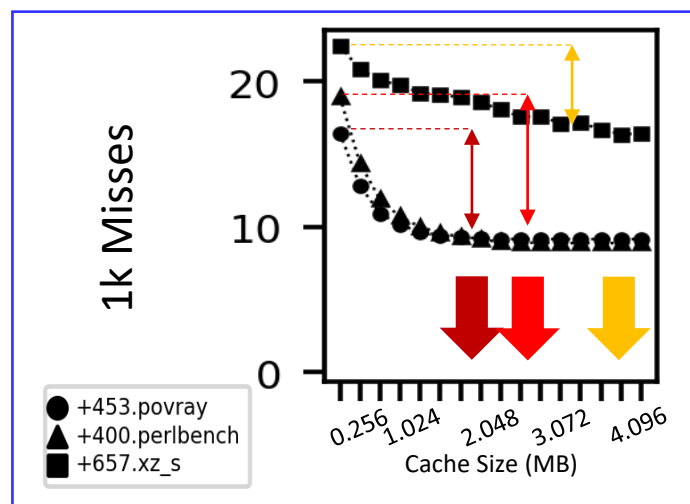
Q: First, what are we looking for?

Annotating a C² to Extract useful Features

Alone



Sharing with named trace

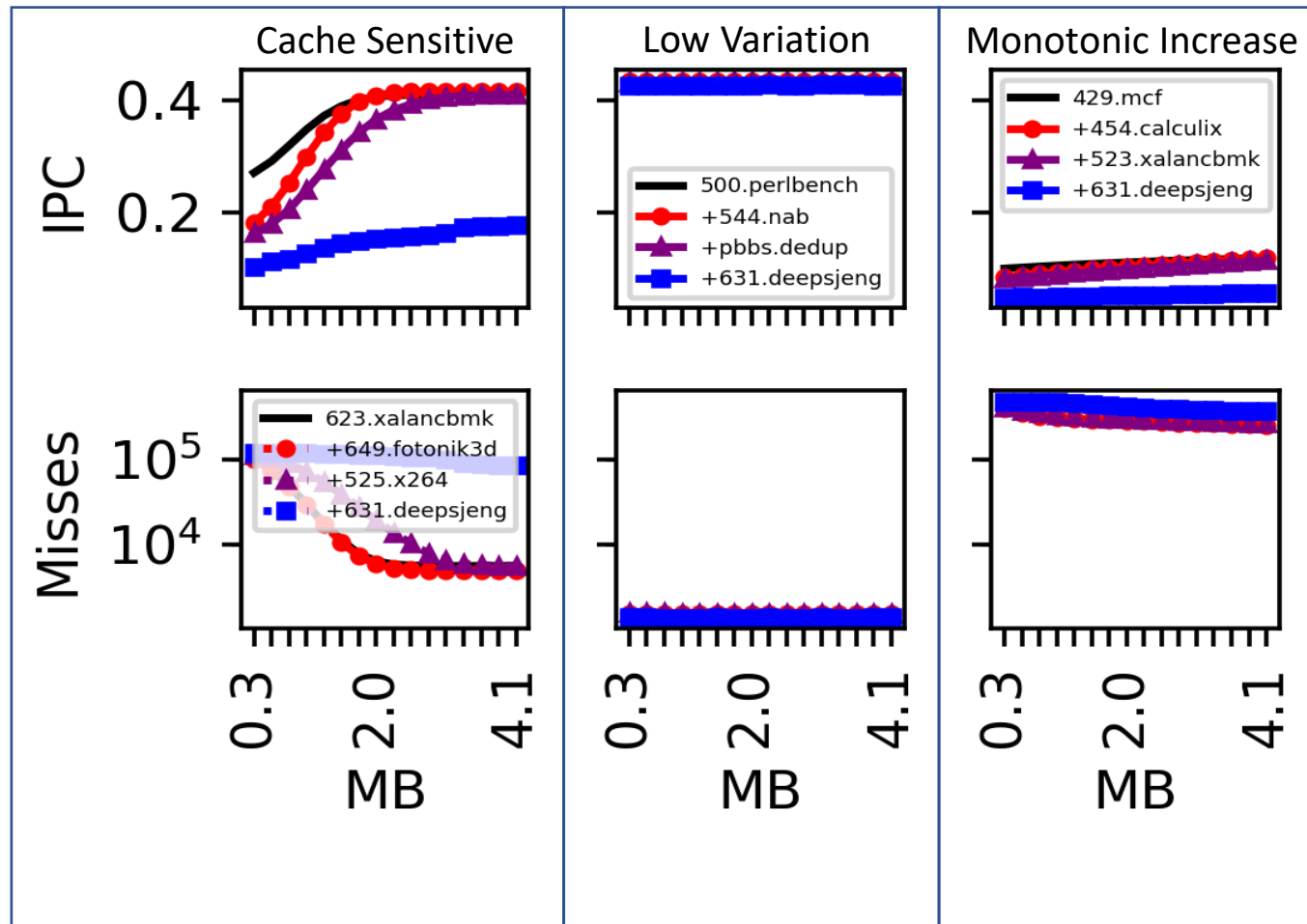


↓ Knee Annotation
↕ Change Annotation

Feature	Definition
IPC Knee (Ik)	$\Delta IPC_{r-s_{Ik}} / IPC_s < 0.01$ $r < s_{Ik}, \forall r, s_{Ik} \in \$ Sizes$
Miss Knee (Mk)	$\Delta Miss_{s_{Mk}-r} / Miss_N < 0.01$ $r < s_{Mk}, \forall r, s_{Mk} \in \$ Sizes$
Knee Difference	$s_{Ik} - s_{Mk}$
Sensitive	Metric changes with \$ Size > 1%
Variating	Vacillations in Curve > 1%
Monotonous	Trend in Curve sans regress (increasing or decreasing)

Q: How do these help to describe behavior?

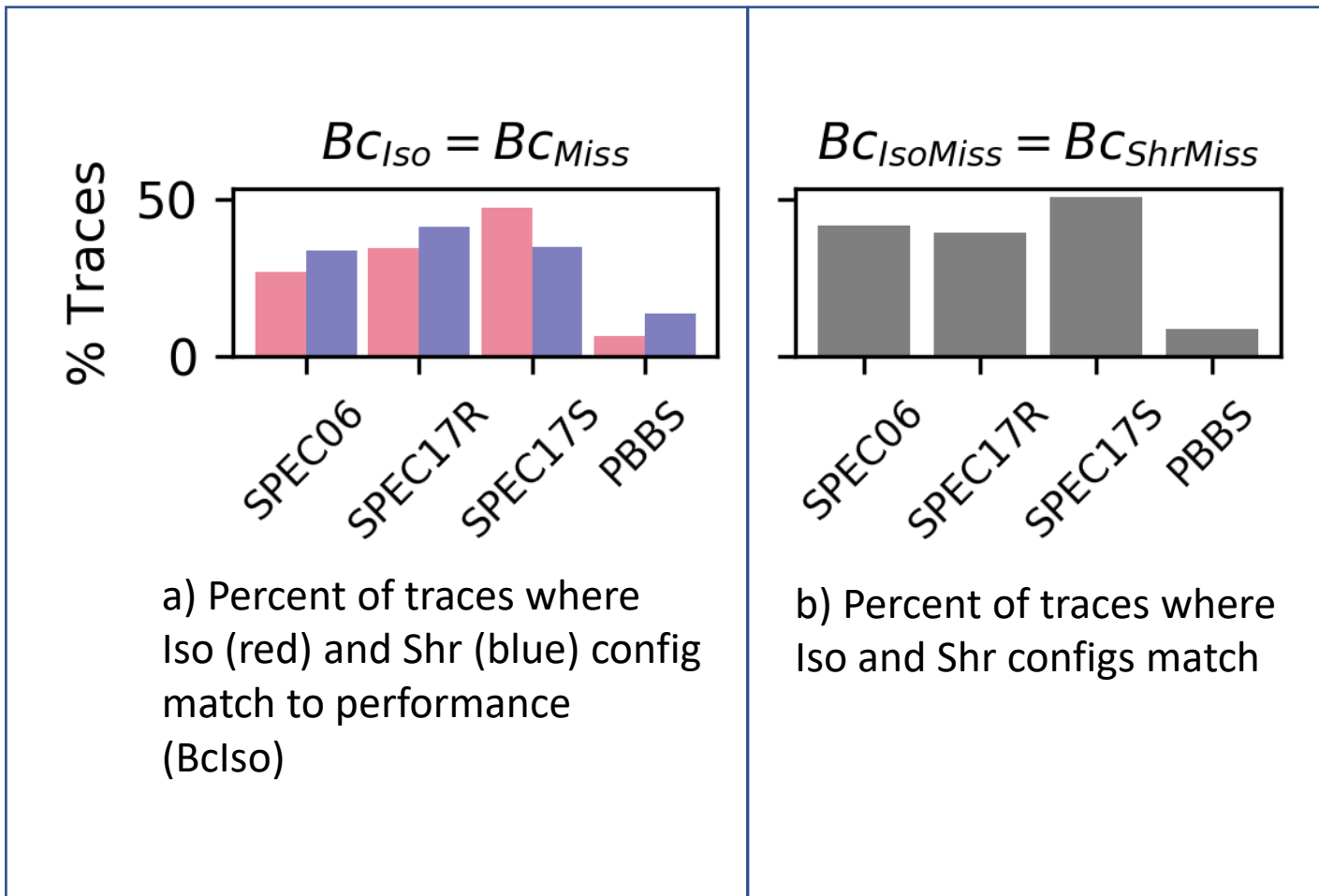
Classifying C² Behavior



Feature	Definition
Cache Sensitive	$\Delta IPC_{4MB-0.2MB} / IPC_{2MB} > 0.01$
Low Variation	$\Delta IPC_{Bc-s} / IPC_{Bc} < 0.01,$ $\forall s \in \$ Sizes$
Monotonic Increase	$IPC_r \leq IPC_s,$ $r < s, \forall r, s \in \$ Sizes$

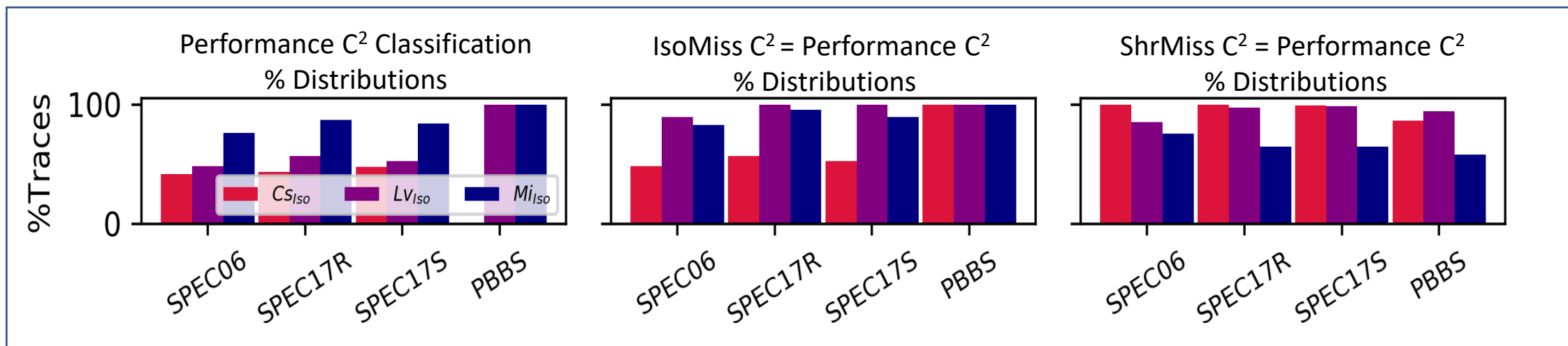
Q: How do the Profiles compare?

Shared Profiles do a better job choosing the best configuration



- Minimizing Isolation Misses ($Bc_{IsoMiss}$) is less likely to match Best Performance Config (Bc_{Iso}) than Minimizing Shared Misses ($Bc_{ShrMiss}$) for all suites except SPEC 17 Speed (figure a).
- $Bc_{IsoMiss}$ and $Bc_{ShrMiss}$ make similar choices between 30-50% of the time with SPEC and 10% of the time with PBBS (figure b).
- **TAKEAWAY:** 25, 20, and 50% increase in config matching supports moving profiling shared behavior; 30% less coverage is explained by parallel programming practices.

Shared Profiles capture cache sensitive and variation behavior



1. Classifications are similar across SPEC, but PBBS is primarily Low Variation and Monotonously Increasing (makes sense since these are core bound).
2. Isolation misses capture performance behavior of workloads that monotonically increase with cache size **BEST**.
3. Shared misses capture workloads that are cache sensitive **BEST**.
4. Both Capture Low Variation behavior comparably well.

TAKEAWAY: Shared profiles capture sensitivity behavior better than isolation profiles while preserving variability estimates of behavior.

In increasingly shared cache hierarchies, we argue estimating cache sensitivity is VERY important.

Conclusion & Future Work

- Cache access behavior is increasingly dominated by contention as core counts scale; consequently, techniques assume preserving isolation behavior informs choices best, particularly with partitioning, but we find isolation misses can **underestimate** needed resources.
- We present **Capacity Curve Annotation and Feature Extraction**, or **C²AFE**, a method for developing capacity curve profiles through curve feature extraction and curve behavior classification.
- We show that shared profiles estimate best performing configurations for more traces than isolation profiles, covering **25, 20, and 50% more traces than isolation profiles**.
- We show that shared profiles **captures important behavior like cache sensitivity better than isolation profiles and captures variation as a function of cache size similarly to isolation profiles**; shared profiles do not capture monotonicity of capacity curves as well.
- Future work includes
 - Directly exploring contention as a function of cache size towards trace behavior classification.
 - Expanding classifications to include curve friendly and curve unfriendly applications in which we analyze how frequently a trace impacts other traces.
 - Reducing profile overheads in cache management techniques like partitioning or insertion policies.

References

- [1] J. Doweck et al. “Inside 6th-Generation Intel Core: New Microarchitecture Code-Named Skylake”. In: IEEE Micro 37.2 (Mar. 2017), pp. 52–62.
- [2] Andrew Herdrich et al. “Cache QoS: From concept to reality in the Intel® Xeon® processor E5-2600 v3 product family”. In: High Performance Computer Architecture (HPCA), 2016 IEEE International Symposium on. IEEE.2016, pp. 657–668.
- [3] Aamer Jaleel. “Memory Characterization of Workloads Using Instrumentation-Driven Simulation A Pin-based Memory Characterization of the SPEC CPU 2000 and SPEC CPU 2006 Benchmark Suites”. In: VSSAD Technical Report 2007.
- [4] Jinchun Kim. ChampSim. <https://github.com/ChampSim/ChampSim>. 2017.
- [5] Anurag Mukkara, Nathan Beckmann, and Daniel Sanchez. “Whirlpool: Improving dynamic cache management with static data classification”. In: ACM SIGARCH Computer Architecture News 44.2 (2016), pp. 113–127.
- [6] A. Sembrant, D. Black-Schaffer, and E. Hagersten. “Phase behavior in serial and parallel applications”. In: 2012 IEEE International Symposium on Workload Characterization (IISWC). Nov. 2012, pp. 47–58.
- [7] Julian Shun et al. “Brief announcement: the problem based benchmark suite”. In: Proceedings of the twenty-fourth annual ACM symposium on Parallelism in algorithms and architectures. ACM. 2012, pp. 68–70.
- [8] Standard Performance Evaluation Corporation. SPEC Benchmark Suite. <http://www.spec.org>.
- [9] Q. Zou et al. “Temporal characterization of SPEC CPU 2006 workloads: Analysis and synthesis”. In: 2012 IEEE 31st International Performance Computing and Communications Conference (IPCCC). Dec. 2012, pp. 11–20.

Acknowledgement

The research presented here was funded by the National GEM Consortium through the reward of their PhD Fellowship.

The data analyzed in this work was generated on the Tufts University High Performance Cluster.